# I302 - Aprendizaje Automático y Aprendizaje Profundo

# Trabajo Práctico 2: Clasificación y Ensemble Learning

Joaquin Szterensus

15 de abril de 2025

Ingeniería en Inteligencia Artificial

# 1. Diagnóstico de Cáncer de Mama

**Resumen**

En este trabajo se intentó clasificar tumores mamarios como benignos o malignos a partir de datos extraidos de imagenes histopatologicas de biopsias mamarias. Para ello, se implementó un modelo de regresión logística binaria con regularización L2 utilizando únicamente `NumPy`. Se entrenaron varios modelos sobre los dos datasets, balanceado y sin balancear, dividiendo los datos en 80 % para entrenamiento y 20 % para validación. En cuanto al dataset sin balancear, se implementaron distintas tecnicas de rebalanceo. El hiperparámetro de regularización $\lambda$ se seleccionó optimizando el F-Score realizando un barrido de este.

## 1.1. Introducción

La entrada del algoritmo consiste en una serie de características cuantitativas y categoricas que describen propiedades celulares (como tamaño, forma, densidad nuclear, tasa de mitosis y presencia de mutaciones), mientras que la salida es una predicción binaria que indica si el tumor es benigno o maligno(0,1).

Antes de entrenar el modelo, se llevó a cabo un análisis exploratorio de los datos para garantizar la calidad de la información. Este proceso permitió limpiar el conjunto de datos, entender la distribución de las variables y detectar valores faltantes u outliers que pudieran afectar el desempeño del modelo.

## 1.2. Métodos

El objetivo fue predecir la probabilidad de que una muestra pertenezca a la clase positiva (por ejemplo, maligno) a partir de un vector de características $\mathbf{x} \in R^n$.

### 1.2.1. Modelo y función de hipótesis:

La hipótesis del modelo se define como:

$$h_\theta(x) = \sigma(z) = \frac{1}{1 + e^{-z}},$$

donde $z$ se calcula como el producto entre el vector de entrada extendido y el vector de parámetros. Es decir, en lugar de escribir

$$z = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n,$$

se puede expresar de forma compacta como:

$$z = x\,(1, x_1, x_2, \ldots, x_n) \cdot \theta\,(\text{bias}, \text{coef}_1, \text{coef}_2, \ldots, \text{coef}_n).$$

Para simplificar la notación, se agrega un término de sesgo (bias) a los datos, lo que equivale a trabajar con un vector extendido

$$\tilde{x} = [1, x_1, \ldots, x_n]^T,$$

y un vector de parámetros

$$\theta = [\text{bias}, \text{coef}_1, \ldots, \text{coef}_n]^T.$$

El objetivo del entrenamiento es ajustar los parámetros $\theta$ de forma que la salida del modelo se acerque lo más posible a las etiquetas reales $y$. La regresión logística con regularización L2 minimiza la siguiente función de costo:

$$J(w) = -\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)}\log(h_w(x^{(i)})) + (1-y^{(i)})\log(1-h_w(x^{(i)}))\right] + \frac{\lambda}{2}\sum_{j=1}^{d} w_j^2 \tag{1}$$

donde $h_w(x) = \sigma(w^T x)$ es la función sigmoide.

- El primer término corresponde al *log-loss* estándar para clasificación binaria.
- El segundo término es la regularización L2, que penaliza grandes valores en los pesos (excepto el sesgo), ayudando a reducir el sobreajuste.
- La suma de regularización comienza en $j = 1$ para excluir el peso del término de sesgo $w_0$.

### 1.2.2. Entrenamiento mediante gradiente descendente:

Para optimizar $J(\theta)$ se utilizó el algoritmo de gradiente descendente. En cada iteración se actualizan los parámetros de la siguiente manera:

$$\theta := \theta - \alpha \nabla J(\theta),$$

donde $\alpha$ es la tasa de aprendizaje y el gradiente se calcula de forma diferenciada para el sesgo y para el resto de los parámetros:

$$\nabla_{\theta_0} J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\big(h_\theta(x^{(i)}) - y^{(i)}\big),$$
$$\nabla_{\theta_j} J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\big(h_\theta(x^{(i)}) - y^{(i)}\big)x_j^{(i)} + \lambda\theta_j, \quad \text{para } j \geq 1.$$

### 1.2.3. Proceso de implementación:

1. **Preprocesamiento:** Se añadió una columna de unos a la matriz de entrada $X$ para incorporar el término de sesgo.
2. **Inicialización:** Los pesos $\theta$ se inicializaron en cero.
3. **Actualización iterativa:** En cada iteración se calculó el valor $z = X\theta$ y se aplicó la función sigmoide para obtener las probabilidades $h_\theta(x)$. Con el error $h_\theta(x) - y$, se computó el gradiente y se actualizó cada componente de $\theta$ utilizando la regla de gradiente descendente.
4. **Predicción:** Para asignar la clase, se aplicó un umbral de 0.5 a la salida del modelo: si $h_\theta(x) \geq 0{,}5$ se predice la clase 1, y 0 en caso contrario.

### 1.2.4. Técnicas de rebalanceo:

Estas técnicas se aplican cuando existe un desbalance significativo entre las clases del conjunto de entrenamiento. En estos casos, es importante rebalancear para evitar que el modelo aprenda a predecir únicamente la clase mayoritaria.

- **Undersampling:** reduce la cantidad de muestras de la clase mayoritaria seleccionando aleatoriamente un subconjunto del mismo tamaño que la clase minoritaria. Si bien puede ser efectivo, tiene como desventaja la posible pérdida de información relevante.
- **Oversampling:** aumenta la cantidad de muestras de la clase minoritaria replicando aleatoriamente observaciones existentes hasta igualar el tamaño de la clase mayoritaria. Puede llevar al sobreajuste si no se aplica con cuidado.
- **Oversampling SMOTE:** (Synthetic Minority Over-sampling Technique) genera nuevas muestras sintéticas de la clase minoritaria interpolando entre observaciones existentes. Esto permite aumentar la representación de la clase minoritaria sin replicar ejemplos exactamente, reduciendo el riesgo de sobreajuste.
- **Re-weighting:** asigna un mayor peso a las observaciones de la clase minoritaria en la función de costo, de modo que el modelo penalice más los errores de clasificación sobre dicha clase. Es útil cuando no se desea modificar el tamaño del conjunto de entrenamiento.

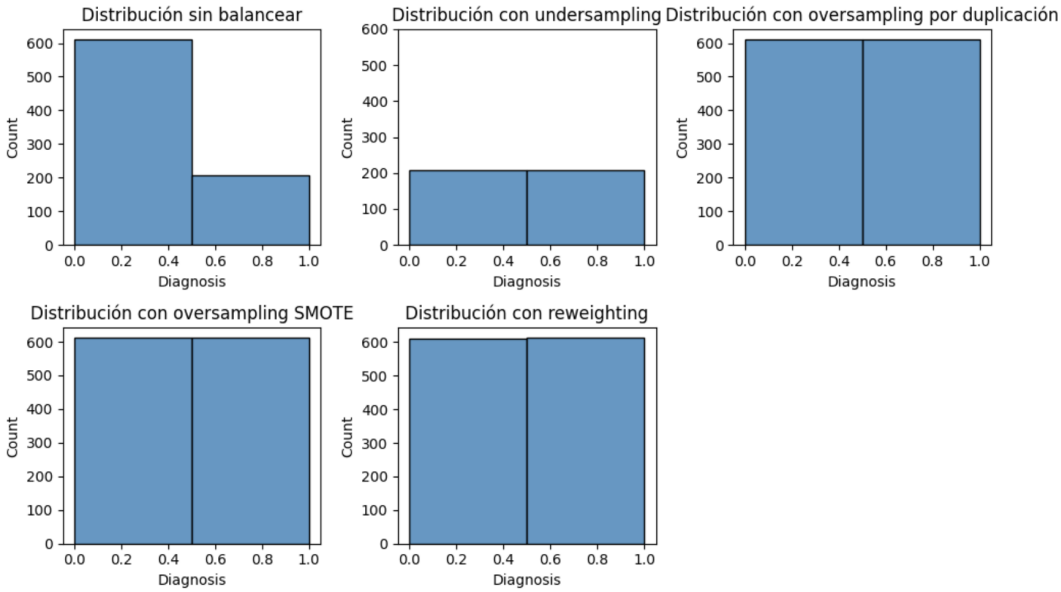


Figura 1: Distribucion de las clases luego de tecnicas de rebalanceo.

### 1.2.5. Proceso de datos:

- **División del conjunto de datos:** se dividieron los datos en un 80 % para entrenamiento y un 20 % para validación. Luego, los valores faltantes (NaN) en cada conjunto fueron rellenados con la mediana correspondiente.

- **Normalización de los datos:** se normalizaron los conjuntos de entrenamiento, validación y prueba utilizando la media y la desviación estándar calculadas a partir del conjunto de entrenamiento, de modo que todos los datos tengan la misma escala.
- **Eliminación de outliers:** se aplicó la técnica del rango intercuartílico (IQR), que consiste en conservar únicamente los datos que se encuentran dentro de un cierto rango porcentual de cada variable, eliminando aquellos que se encuentran fuera de dicho rango. Como se observa en la Figura 2, la mayoría de los datos se concentran en un rango limitado; los valores que se alejan significativamente de este rango probablemente correspondan a errores de medición o registros atípicos.

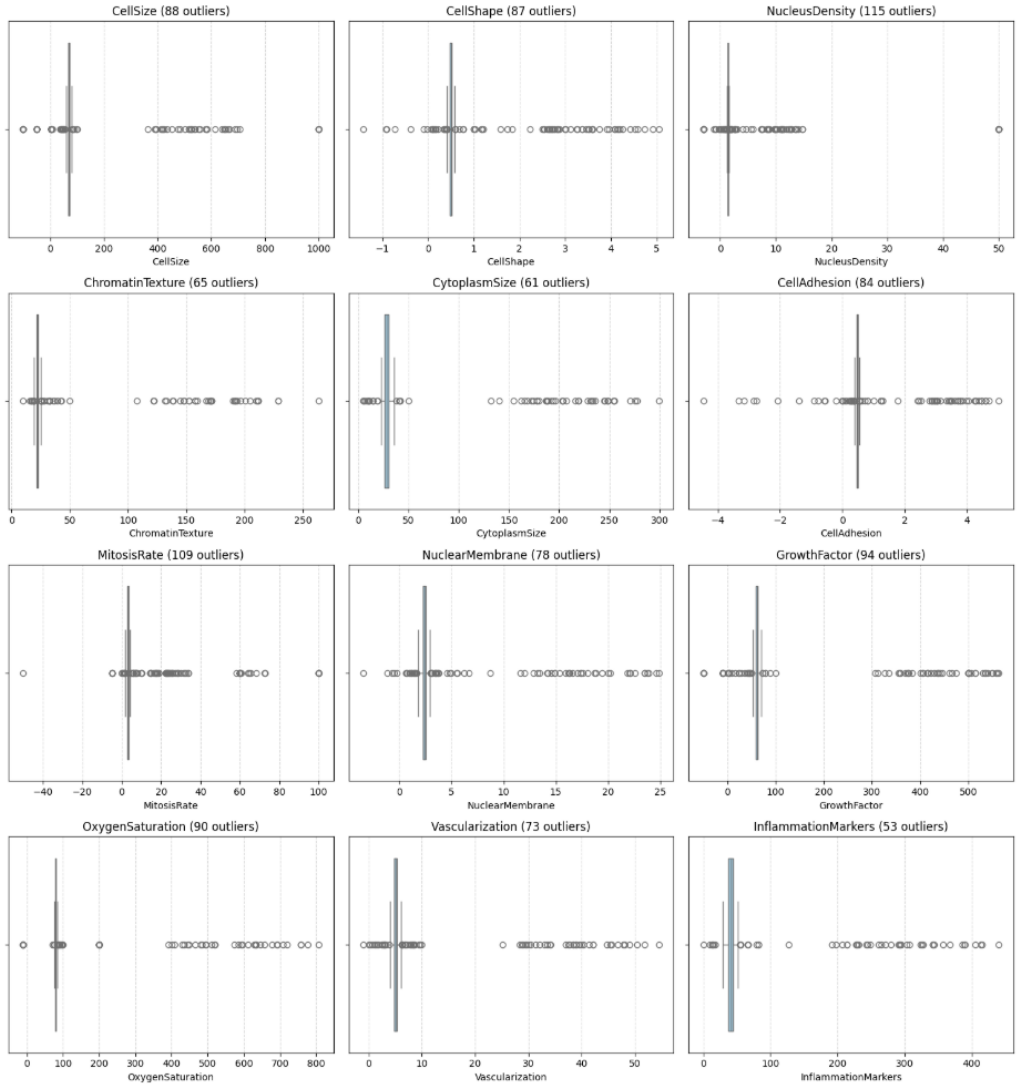


Figura 2: Distribución de los datos antes de eliminar outliers utilizando la técnica del IQR.

**Métricas de clasificación:**

- **Accuracy**: $\frac{1}{n}\sum_{i=1}^{n} I(y_i = \hat{y}_i)$
- **Precisión**: $P = \frac{TP}{TP+FP}$
- **Recall (sensibilidad)**: $R = \frac{TP}{TP+FN}$
- **F1-Score**: $F_1 = 2 \cdot \frac{P \cdot R}{P+R}$
- **AUC-ROC**: $\int_0^1 TPR(FPR)\, dFPR$
- **AUC-PR**: $\int_0^1 P(R)\, dR$

Donde: $TP$ = verdaderos positivos, $FP$ = falsos positivos, $FN$ = falsos negativos, $TPR = \frac{TP}{TP+FN}$, $FPR = \frac{FP}{FP+TN}$.

Los valores de $\lambda$ se generaron en escala logarítmica:

$$\lambda \in \left\{10^x \mid x \in \{-4, -3.\overline{1}, -2.\overline{2}, \ldots, 1\}\right\}$$

es decir,

$$\lambda_i = 10^{-4+\frac{5}{9}(i-1)} \quad \text{para } i = 1, \ldots, 10$$

obteniendo así 10 valores equidistantes en escala logarítmica entre $10^{-4}$ y $10^1$. Finalmente, se realizó un barrido sobre estos valores seleccionando aquel que maximiza el **F1-Score**.

## 1.3. Resultados

Para el dataset balanceado, el mejor lambda para la regularizacion fue 0.0001 con un F1-SCORE de 0.87 del conjunto de validacion. Al evaluar sobre el conjunto de prueba, sus metricas son las siguientes:

| **Modelo** | **Accuracy** | **Precision** | **Recall** | **F1-Score** | **AUC-ROC** | **AUC-PR** |
|---|---|---|---|---|---|---|
| Balanceado | 0.88 | 0.86 | 0.86 | 0.86 | 0.80 | 0.69 |

Cuadro 1: Métricas de clasificación para modelo entrenad con datos balanceados y lambda=0.0001.

con su matriz de confusion

| | **Predicción** | |
|---|---|---|
| **Real** | **Clase 0** | **Clase 1** |
| **Clase 0** | 91 | 11 |
| **Clase 1** | 11 | 72 |

Cuadro 2: Matriz de confusión

| Modelo | Accuracy | Precision | Recall | F1-Score | AUC-ROC | AUC-PR |
|---|---|---|---|---|---|---|
| Sin Balancear | 0.766 | 0.50 | 0.03 | 0.06 | 0.85 | 0.61 |
| Undersampling | 0.881 | 0.74 | 0.75 | 0.74 | 0.85 | 0.66 |
| Oversampling | 0.881 | 0.73 | 0.77 | 0.75 | 0.85 | 0.66 |
| Oversamplig SMOTE | 0.885 | 0.74 | 0.77 | 0.75 | 0.85 | 0.65 |
| Re-Weighting | 0.889 | 0.73 | 0.82 | 0.77 | 0.86 | 0.66 |

Cuadro 3: Métricas de clasificación para modelos entrenados con datos balanceados luego de tecnicas de rebalanceo y sus respectivos lambdas.

**Dataset Rebalanceado:**

Los resultados muestran que la aplicación de técnicas de rebalanceo es fundamental para mejorar la capacidad del modelo de detectar la clase minoritaria. Mientras que el modelo sin rebalancear presenta métricas inaceptables para esta clase, los métodos basados en *undersampling*, *oversampling* y *SMOTE* logran un desempeño similar en términos de *accuracy*, *precision* y *AUC*, pero es el método de *Re-Weighting* el que ofrece la mejor combinación entre *recall* y *F1-Score*.

Dado que en un entorno de producción es importante minimizar los falsos negativos (por ejemplo, en contextos donde pasar por alto una instancia positiva puede tener consecuencias costosas o peligrosas), se prioriza un modelo con alto *recall*. Además, el hecho de que *Re-Weighting* mantenga valores competitivos en *accuracy*, *precision*, *AUC-ROC* y *AUC-PR* lo convierte en un modelo robusto y confiable.

Por lo tanto, el modelo entrenado con la técnica de Re-Weighting sería el elegido para implementar en producción, ya que ofrece un buen equilibrio entre sensibilidad y precisión, sin sacrificar capacidad discriminativa general. Esta elección se justifica tanto desde una perspectiva métrica como desde una evaluación práctica del impacto de los errores.

# 2. Predicción de Rendimiento de Jugadores de Basketball

**Resumen**

En este trabajo se buscó modelar la clase de rendimiento de jugadores de basketball a partir de sus estadísticas, utilizando un enfoque de clasificación multiclase. Para ello, se realizó un preprocesamiento del conjunto de datos, incluyendo la limpieza, la normalización y la división en conjuntos de entrenamiento y validación. Se entrenaron y compararon tres modelos distintos: regresión logística multiclase, análisis discriminante lineal (LDA) y random forest. La evaluación se llevó a cabo utilizando distintos tipos de metricas.

## 2.1. Introducción

El problema abordado consiste en clasificar el rendimiento de jugadores de basket en diferentes categorías, denominadas `war_class`, a partir de un conjunto de variables estadísticas. La entrada del algoritmo está compuesta por diversas mediciones numéricas relacionadas con el desempeño de los jugadores, extraídas del conjunto de datos original. Entre estas variables se incluyen indicadores de rendimiento y otras métricas relevantes, las cuales se han preprocesado y normalizado para su análisis.

Para obtener la predicción de la categoría de rendimiento (la salida del algoritmo), se implementó un modelo de regresión logística multiclase, complementado con otros modelos comparativos como el análisis discriminante lineal (LDA) y el random forest. Estos modelos fueron entrenados utilizando una división de los datos en conjuntos de entrenamiento y validación, permitiendo evaluar su desempeño mediante métricas .

## 2.2. Métodos

Se desarrollaron e implementaron tres modelos para abordar la tarea de clasificación del rendimiento de jugadores: **L2RegresiónMulticlase**, **Análisis Discriminante Lineal (LDA)** y **Random Forest**.

### 2.2.1. clases

- **L2RegresiónMulticlase:** Este modelo utiliza la regresión logística para la clasificación multiclase. Sea $\mathbf{X} \in R^{n \times p}$ la matriz de características y $y \in \{c_1, \ldots, c_K\}$ el vector de etiquetas, el modelo estima probabilidades mediante la función *softmax*:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)},$$

  con $z = \mathbf{X}\mathbf{W} + \mathbf{b}$, donde $\mathbf{W} \in R^{p \times K}$ y $\mathbf{b} \in R^{1 \times K}$ son los parámetros a estimar. Además, se incorpora una regularización L2, cuyo término de penalización es:

$$\frac{\lambda}{2} \|\mathbf{W}\|_2^2,$$

  lo que suaviza la solución y previene el sobreajuste.

- **Análisis Discriminante Lineal (LDA):** LDA asume que los datos de cada clase siguen una distribución normal con la misma matriz de covarianza, y se basa en la función discriminante:

$$g_c(\mathbf{x}) = \mathbf{x}^T \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^T \Sigma^{-1} \mu_c + \ln(\pi_c),$$

  donde $\mu_c$ es el vector de media de la clase $c$, $\Sigma$ es la matriz de covarianza común y $\pi_c$ la probabilidad a priori de la clase $c$. La predicción se realiza asignando a la muestra la clase para la cual $g_c(\mathbf{x})$ es máximo.

- **Random Forest:** Este algoritmo es un método de ensamble que construye múltiples árboles de decisión a partir de muestras bootstrap del conjunto de datos. Cada árbol realiza una predicción y la clase final se determina por votación mayoritaria. Los árboles individuales se entrenan utilizando particiones de las características en cada nodo, evaluando múltiples umbrales para lograr la máxima ganancia de información.

#### Conjunto de Datos y Preprocesamiento

El conjunto de datos utilizado contiene variables numéricas relacionadas con el rendimiento de los jugadores y la clase objetivo `war_class`. Al analizar los datos, se observó que el

dataset no presentaba valores faltantes y se encontraba bastante balanceado, como se aprecia en la Figura 4. Únicamente se eliminó la columna `war_total`, ya que presentaba una correlación excesivamente alta con la variable objetivo, lo que provocaba que todos los modelos funcionen de manera perfecta sin poder distinguir el desempeño real.Esto puede notarse en la Figura 3 Finalmente, se dividió el set de desarrollo en 80 % para entrenamiento y 20 % para validación, para posteriormente normalizar los datos utilizando la media y la desviación estándar calculadas en el conjunto de entrenamiento, de modo que todos los datos se ajusten al mismo escalado.

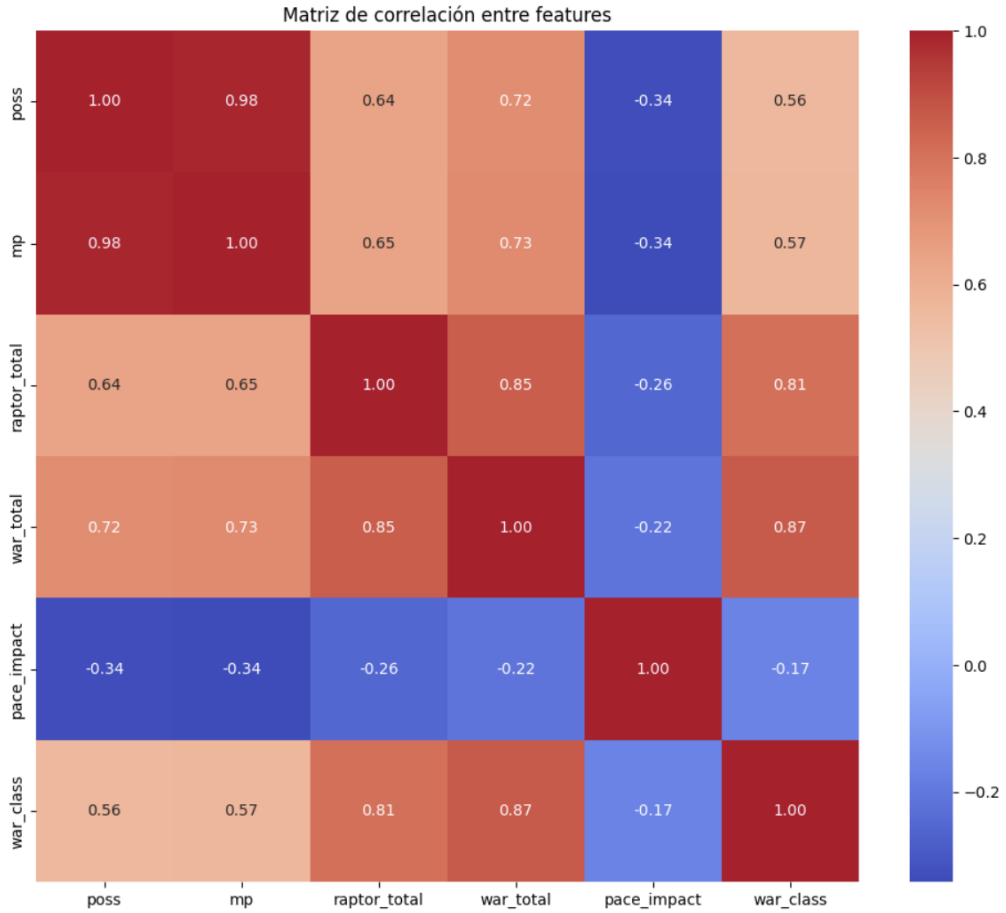


Figura 3: Matriz de correlación entre las variables. Se observa una relación muy alta entre `war_total` y la variable objetivo (`war_class`).

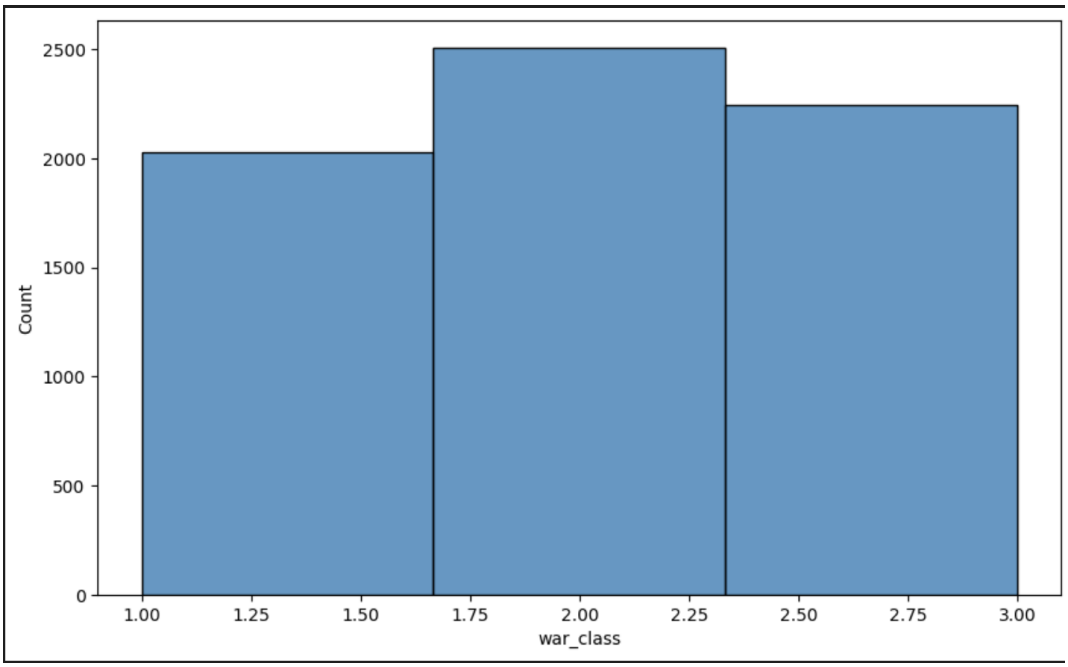


Figura 4: Cantidad de muestra de las distintas clases de la variable objetivo (`war_class`).

### Selección de Hiperparámetros y Validación

Para la configuración de los modelos se seleccionaron diversos hiperparámetros:

- **L2RegresiónMulticlase:** Se definieo el coeficiente de regularización ($\lambda$).
- **Random Forest:** Se determinaron el número de árboles (*n_estimators*) y la profundidad máxima (max_depth).

Los hiperparametros de ambos modelos se consiguieron realizando un barridos de lambdas buscando maximizar la metrica F1-SCORE, utilizando el conjunto de validacion.

estos parametros terminaron siendo 15 estimadores con 5 de profundiad para el random forest con un F1-SCORE=0.97 y ($\lambda$)= 0.0001 para la regresion logistica con un F1-SCORE=0.88

### Métricas de Rendimiento Utilizadas

En este ejercicio, al tratarse de un problema de clasificación multiclase, se adoptó la técnica *one versus rest* para evaluar el desempeño de los modelos. En esta tecnica, se considera cada clase individualmente como la clase positiva, mientras que el resto de las clases se agrupan en una sola categoría negativa. Esto permite calcular métricas como precisión, recall y F1-score para cada clase de forma individual, las cuales posteriormente son promediadas para obtener una medida global del rendimiento. Además, se complementó el análisis con la utilización de las curvas ROC/PR adaptadas a escenarios multiclase, facilitando una comparación integral entre los distintos modelos.

## 2.3. Resultados

En esta sección se presentan y discuten los resultados obtenidos por los tres modelos evaluados: **LDA**, **Regresión Logística Multiclase** (One-vs-Rest) y **Random Forest**. La Tabla 4 resume las métricas más relevantes calculadas sobre el conjunto de test:

| **Modelo** | **Accuracy** | **Precision** | **Recall** | **F1-Score** | **AUC-ROC** | **AUC-PR** |
|---|---|---|---|---|---|---|
| LDA | 0.90 | 0.86 | 0.91 | 0.90 | 0.96 | 0.91 |
| Reg. Logística | 0.87 | 0.88 | 0.88 | 0.87 | 0.94 | 0.89 |
| Random Forest | 0.95 | 0.91 | 0.95 | 0.95 | 0.98 | 0.12 |

Cuadro 4: Métricas de rendimiento para los distintos modelos sobre el conjunto de test.

Como se puede observar, el modelo que alcanzó el mejor desempeño en la mayoría de las métricas fue **Random Forest**, destacándose en *accuracy*, *recall* y *F1-score*, lo cual sugiere una excelente capacidad para capturar patrones complejos en los datos.

Los modelos de **LDA** y **Regresión Logística** también presentaron un rendimiento competitivo, especialmente en términos de *AUC-ROC* y *AUC-PR*, mostrando un comportamiento estable y balanceado entre clases. Estos resultados se ven reflejados en las respectivas curvas ROC y Precision-Recall.

A continuación se presentan las curvas ROC y Precision-Recall para cada modelo:

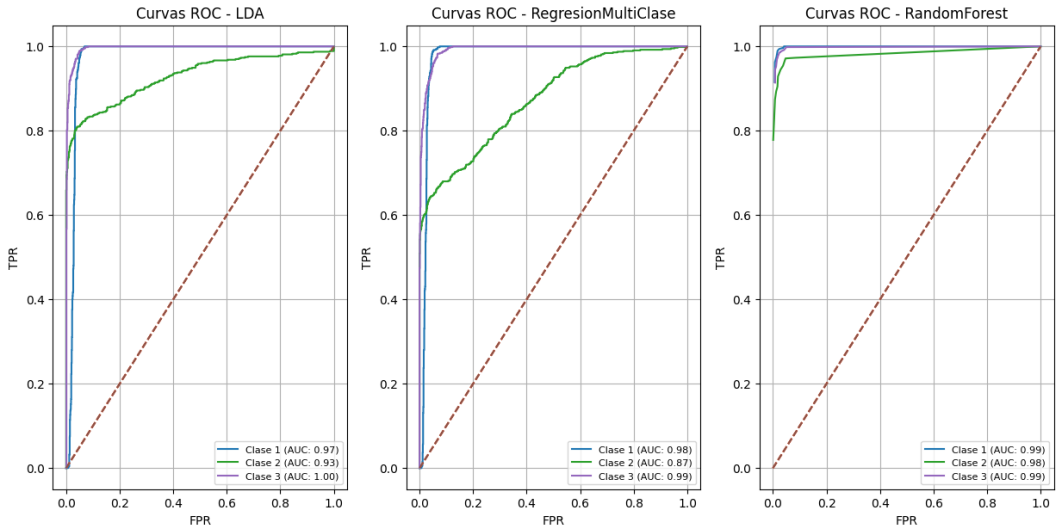


Figura 5: Curvas ROC por clase para los modelos LDA, Regresión Logística Multiclase y Random Forest.

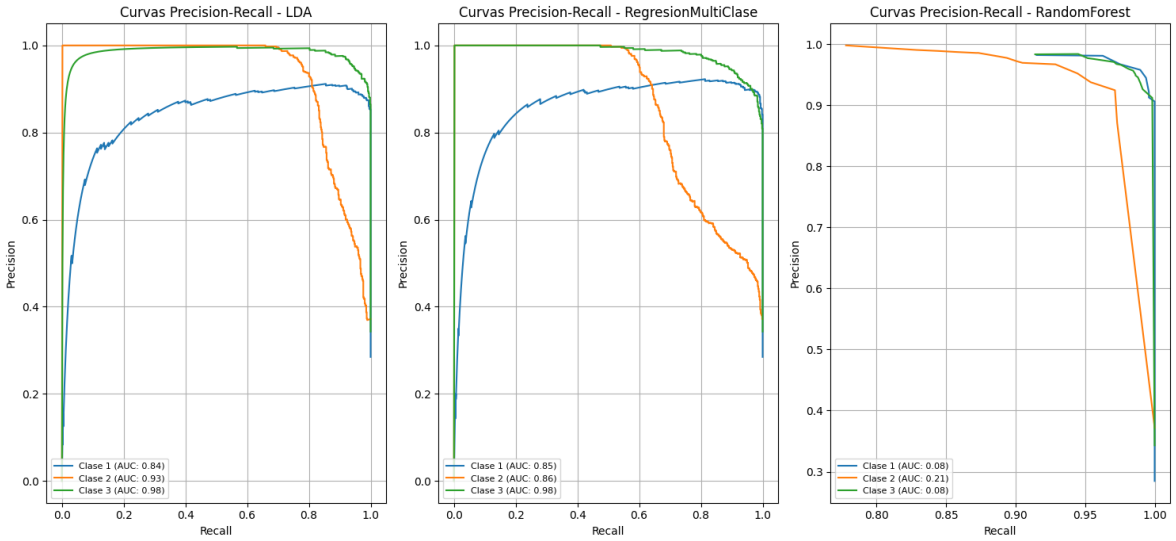


Figura 6: Curvas Precision-Recall por clase para los modelos LDA, Regresión Logística Multiclase y Random Forest.

En base a las métricas obtenidas y la interpretación de las curvas, el modelo más recomendable para llevar a producción sería **Random Forest**, ya que muestra un desempeño destacado en todas las métricas clave. Su alta capacidad de generalización, sumada a su robustez frente a la variabilidad de los datos, lo posicionan como la mejor opción entre los modelos evaluados.